

Name: Eman Fatima
Reg No: BSSE-23S-0025
Program: BSSE
Date: August 27th, 2026

WEEK NO. 2-

PART A: C# INTERMEDIATE BASICS

Exercises:-

1. Create a Generic Helper Method

Code:

```
PartA-CSharp-Intermediate > Helpers > C# GenericHelper.cs > {} Week2.PartA.Helpers > GenericHelper
1  namespace Week2.PartA.Helpers;
2
3  3 references
4  public static class GenericHelper
5  {
6      3 references
7      public static void DisplayValue<T>(T value)
8      {
9          Console.WriteLine($"Value: {value}");
10     }
11 }
```

```
PartA-CSharp-Intermediate > Program.cs
7  Console.WriteLine("=====");
8  Console.WriteLine("EXERCISE 1 - GENERIC HELPER METHOD");
9  Console.WriteLine("=====");
10
11 GenericHelper.DisplayValue<int>(100);
12 GenericHelper.DisplayValue<string>("Hello C#");
13 GenericHelper.DisplayValue<double>(85.5);
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 2\PartA-CSharp-Intermediate> dotnet run
=====
EXERCISE 1 - GENERIC HELPER METHOD
=====
Value: 100
Value: Hello C#
Value: 85.5
```

2. Create an Interface with Two Implementations

Code:

```
PartA-CSharp-Intermediate > Notifications > INotificationService.cs > {} Week2.PartA.Notifications > INotificationService
1  namespace Week2.PartA.Notifications;
2
3  4 references
4  public interface INotificationService
5  {
6      4 references
7      void SendMessage(string message);
8  }
```

```

PartA-CSharp-Intermediate > Notifications > EmailNotifier.cs > {} Week2.PartA.Notifications > EmailNotifier
1  namespace Week2.PartA.Notifications;
2
3  1 reference
4  public class EmailNotifier : INotificationService
5  {
6      3 references
7      public void SendMessage(string message)
8      {
9          Console.WriteLine($"Email sent: {message}");
10     }
11 }

```

```

PartA-CSharp-Intermediate > Notifications > SmsNotifier.cs > {} Week2.PartA.Notifications > SmsNotifier
1  namespace Week2.PartA.Notifications;
2
3  1 reference
4  public class SmsNotifier : INotificationService
5  {
6      3 references
7      public void SendMessage(string message)
8      {
9          Console.WriteLine($"SMS sent: {message}");
10     }
11 }

```

```

PartA-CSharp-Intermediate > Program.cs
15
16 Console.WriteLine();
17 Console.WriteLine("=====");
18 Console.WriteLine("EXERCISE 2 - INTERFACE");
19 Console.WriteLine("=====");
20
21 INotificationService emailNotifier = new EmailNotifier();
22 emailNotifier.SendMessage("Welcome to the internship.");
23
24 INotificationService smsNotifier = new SmsNotifier();
25 smsNotifier.SendMessage("Your registration is complete.");

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 2\PartA-CSharp-Intermediate> dotnet run
=====
EXERCISE 2 - INTERFACE
=====
Email sent: Welcome to the internship.
SMS sent: Your registration is complete.

```

3. Write an Async Method That Waits and Returns a Result

Code:

```

PartA-CSharp-Intermediate > AsyncDemo > C# ResultService.cs > { } Week2.PartA.AsyncDemo > ResultService
1  namespace Week2.PartA.AsyncDemo;
2
3  2 references
4  public class ResultService
5  {
6      1 reference
7      public async Task<string> GetResultAsync()
8      {
9          Console.WriteLine("Processing result...");
10
11         await Task.Delay(2000);
12
13         return "Result received successfully.";
14     }
15 }

```

```

PartA-CSharp-Intermediate > C# Program.cs
28 Console.WriteLine();
29 Console.WriteLine("=====");
30 Console.WriteLine("EXERCISE 3 - ASYNC/AWAIT");
31 Console.WriteLine("=====");
32
33 ResultService resultService = new ResultService();
34
35 string result = await resultService.GetResultAsync();
36
37 Console.WriteLine(result);

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 2\PartA-CSharp-Intermediate> dotnet run
=====
EXERCISE 3 - ASYNC/AWAIT
=====
Processing result...
Result received successfully.

```

4. Refactor a Messy Console Program into Smaller Methods/Classes

Code:

PartA-CSharp-Intermediate > RefactorDemo > C# Student.cs > {} Week2.PartA.RefactorDemo > 🐞 Student

```
1 namespace Week2.PartA.RefactorDemo;
2
3 4 references
4 public class Student
5 {
6     2 references
7     public string Name { get; set; }
8     3 references
9     public int Marks { get; set; }
10
11     1 reference
12     public Student(string name, int marks)
13     {
14         Name = name;
15         Marks = marks;
16     }
17 }
```

PartA-CSharp-Intermediate > RefactorDemo > C# GradeCalculator.cs > {} Week2.PartA.RefactorDemo > 🐞 GradeCalculator

```
1 namespace Week2.PartA.RefactorDemo;
2
3 4 references
4 public class GradeCalculator
5 {
6     1 reference
7     public string GetGrade(int marks)
8     {
9         if (marks >= 80)
10             return "A";
11         if (marks >= 70)
12             return "B";
13         if (marks >= 60)
14             return "C";
15         if (marks >= 50)
16             return "D";
17         return "F";
18     }
19 }
20
21 }
```

```

PartA-CSharp-Intermediate > RefactorDemo > StudentReportService.cs > {} Week2.PartA.RefactorDemo > StudentReportService
1  namespace Week2.PartA.RefactorDemo;
2
3  3 references
4  public class StudentReportService
5  {
6      2 references
7      private readonly GradeCalculator _gradeCalculator;
8
9      1 reference
10     public StudentReportService(GradeCalculator gradeCalculator)
11     {
12         _gradeCalculator = gradeCalculator;
13     }
14
15     1 reference
16     public void PrintReport(Student student)
17     {
18         string grade = _gradeCalculator.GetGrade(student.Marks);
19
20         Console.WriteLine($"Student: {student.Name}");
21         Console.WriteLine($"Marks: {student.Marks}");
22         Console.WriteLine($"Grade: {grade}");
23     }
24 }

```

PartA-CSharp-Intermediate > Program.cs

```

40 Console.WriteLine();
41 Console.WriteLine("=====");
42 Console.WriteLine("EXERCISE 4 - CLEAN CODE REFACTOR");
43 Console.WriteLine("=====");
44
45 Student student = new Student("Ali", 85);
46
47 GradeCalculator gradeCalculator = new GradeCalculator();
48
49 StudentReportService reportService =
50     new StudentReportService(gradeCalculator);
51
52 reportService.PrintReport(student);

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 2\PartA-CSharp-Intermediate> dotnet run
=====
EXERCISE 4 - CLEAN CODE REFACTOR
=====
Student: Ali
Marks: 85
Grade: A
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 2\PartA-CSharp-Intermediate>

```

PART B: ASP.NET CORE FUNDAMENTALS

Exercises:-

1. Create an ASP.NET Core API With One GET Endpoint

Code:

PartB-ASPNetCore-Basics > Models >  Student.cs > {} PartB_ASPNetCore_Basics.Models >  Student

```
1 namespace PartB_ASPNetCore_Basics.Models;
2
3 5 references
4 public class Student
5 {
6     7 references
7     public int Id { get; set; }
8
9     4 references
10    public string Name { get; set; } = string.Empty;
11
12    2 references
13    public string Department { get; set; } = string.Empty;
14 }
```

PartB-ASPNetCore-Basics > Controllers >  StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers >  StudentsController

```
1 using Microsoft.AspNetCore.Mvc;
2 using PartB_ASPNetCore_Basics.Models;
3
4 namespace PartB_ASPNetCore_Basics.Controllers;
5
6 [ApiController]
7 [Route("api/[controller]")]
```

PartB-ASPNetCore-Basics > Controllers >  StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers >  StudentsController >

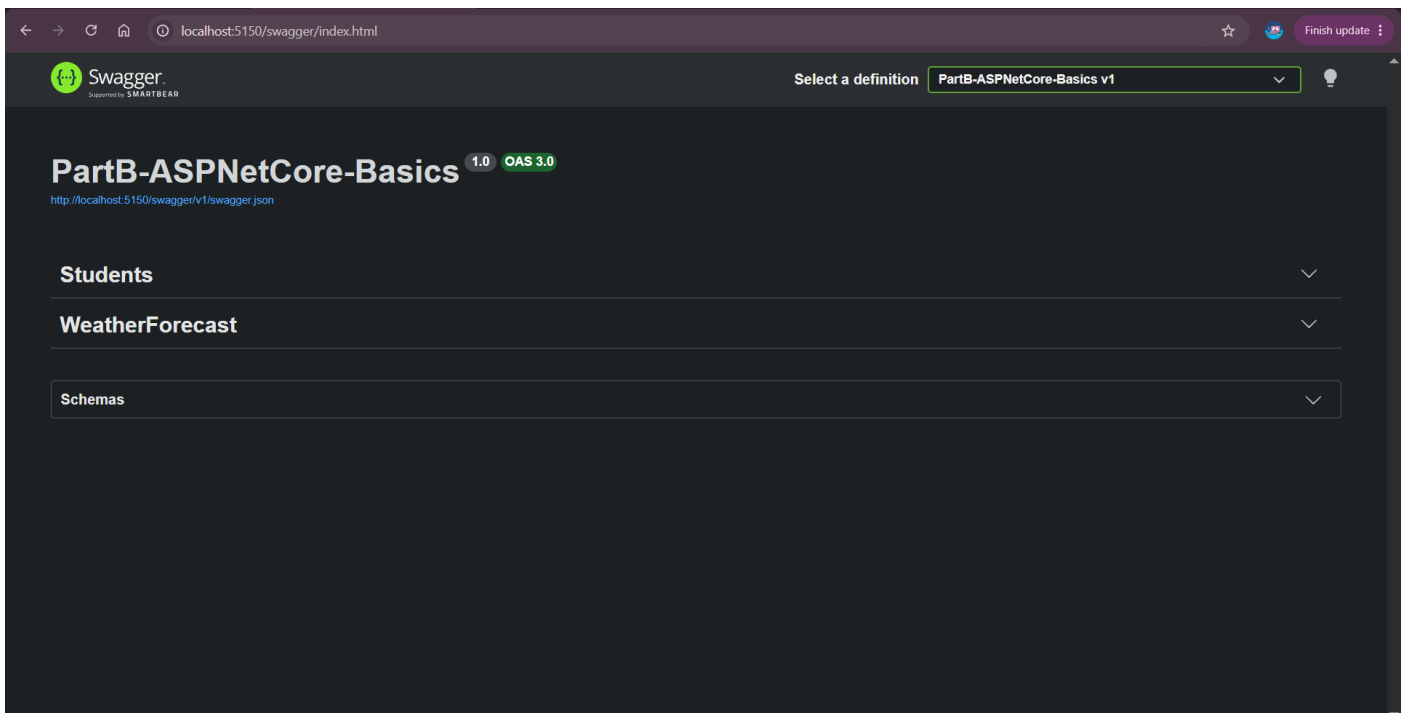
```
4 namespace PartB_ASPNetCore_Basics.Controllers;
8 public class StudentsController : ControllerBase
36 [HttpGet]
37 0 references
38 public IActionResult GetStudents()
39 {
40     _logger.LogInformation("Getting all students.");
41     return Ok(Students);
42 }
```

```

PartB-ASPNetCore-Basics > C# Program.cs
1  var builder = WebApplication.CreateBuilder(args);
2
3  builder.Services.AddControllers();
4
5  builder.Services.AddEndpointsApiExplorer();
6
7  builder.Services.AddSwaggerGen();
8
9  var app = builder.Build();
10
11 if (app.Environment.IsDevelopment())
12 {
13     app.UseSwagger();
14
15     app.UseSwaggerUI();
16 }
17
18 app.UseHttpsRedirection();
19
20 app.MapControllers();
21
22 app.Run();

```

Output:



localhost:5150/swagger/index.html

Swagger
Supported by SMARTBEAR

Select a definition **PartB-ASPNetCore-Basics v1**

PartB-ASPNetCore-Basics ^{1.0} ^{OAS 3.0}

<http://localhost:5150/swagger/v1/swagger.json>

Students

- GET** /api/Students
- POST** /api/Students
- GET** /api/Students/{id}

WeatherForecast

- GET** /WeatherForecast

Schemas

Student {

- id > [...]
- name > [...]
- department > [...]

}

WeatherForecast {

- date > [...]
- temperatureC > [...]
- temperatureF > [...]
- summary > [...]

}

Student {

- id > [...]
- name > [...]
- department > [...]

}

WeatherForecast {

- date > [...]
- temperatureC > [...]
- temperatureF > [...]
- summary > [...]

}

localhost:5150/swagger/index.html

Students

GET /api/Students

Parameters

No parameters

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5150/api/Students' \
-H 'accept: */*'
```

Request URL

<http://localhost:5150/api/Students>

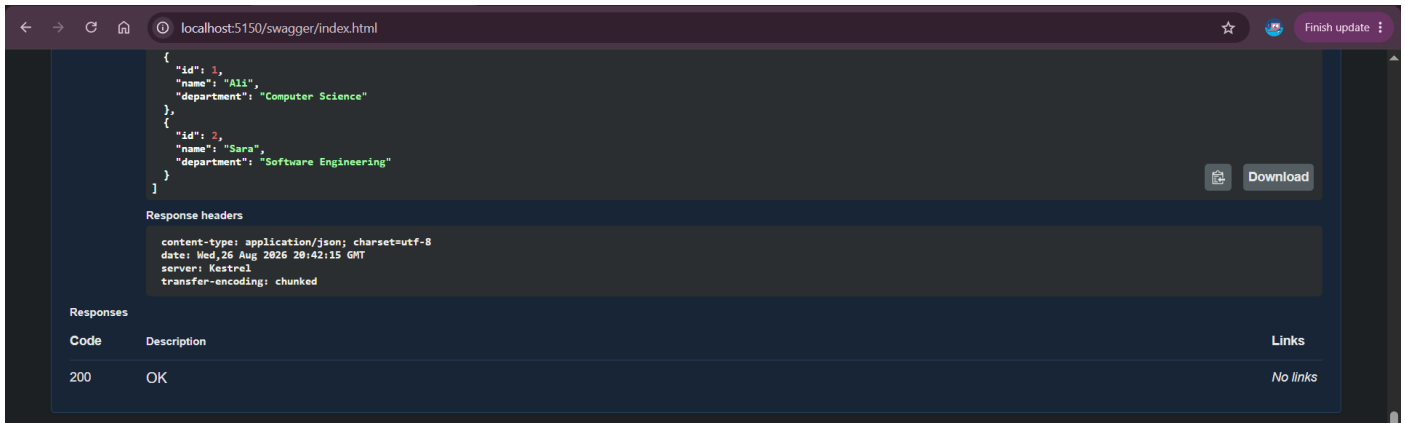
Server response

Code **Details**

200

Response body

```
[
  {
    "id": 1,
    "name": "Ali",
    "department": "Computer Science"
  },
  {
    "id": 2,
```



2. Create a Second Endpoint That Accepts Input

Code:

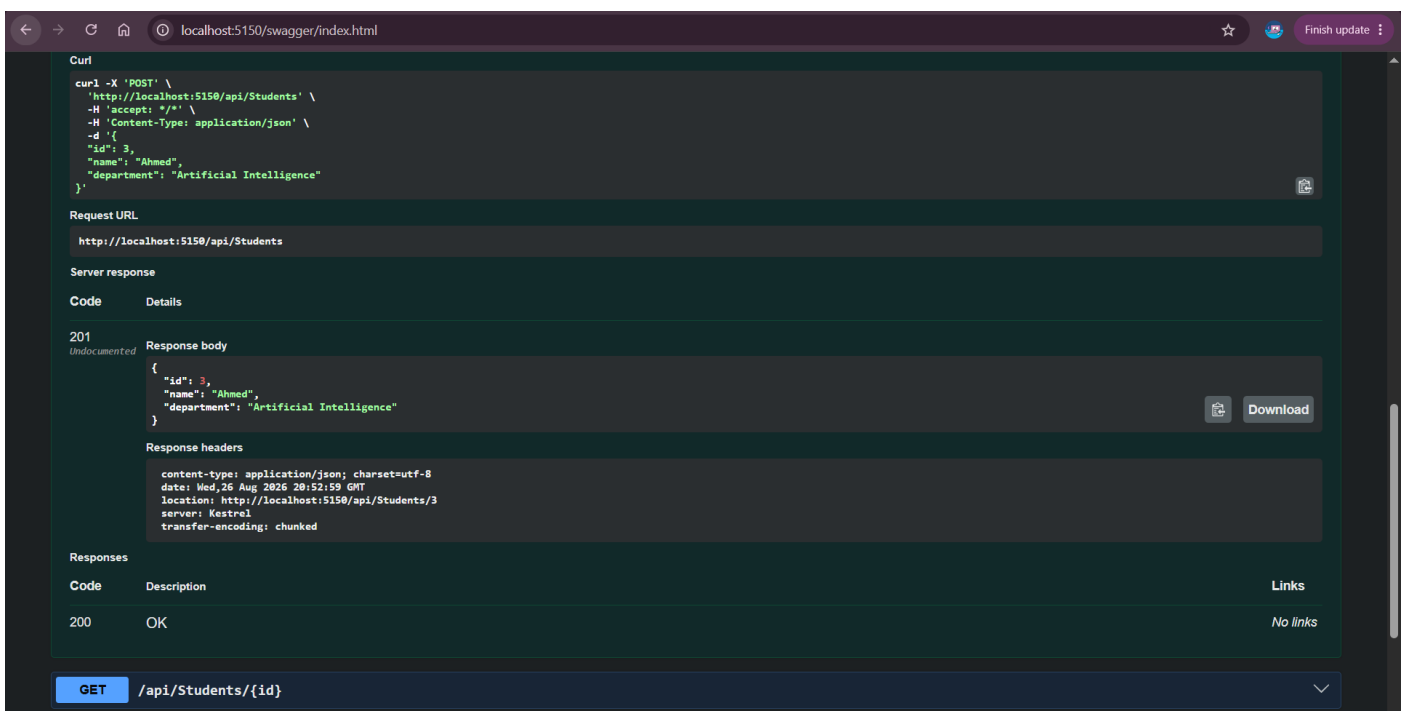
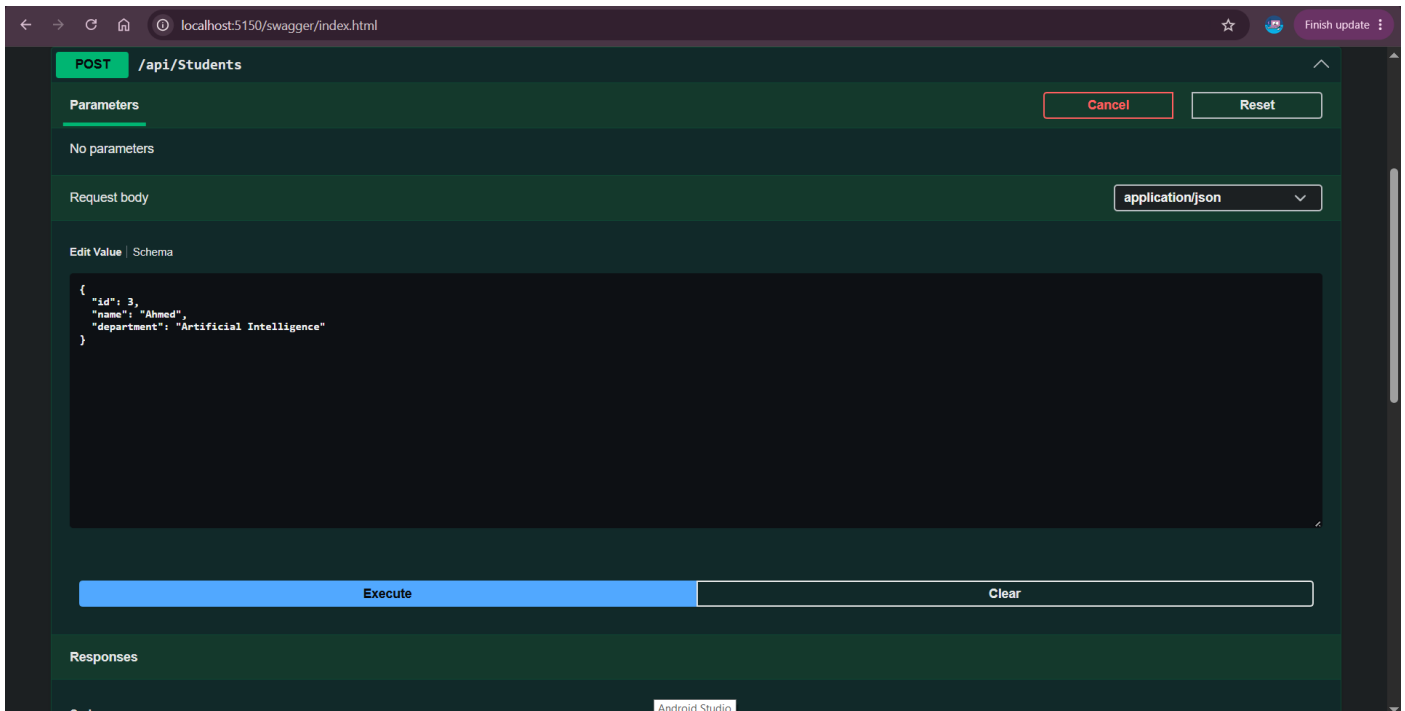
```
PartB-ASPNetCore-Basics > Controllers > StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers > StudentsController
4 namespace PartB_ASPNetCore_Basics.Controllers;
8 public class StudentsController : ControllerBase
67 [HttpPost]
   0 references
68 public IActionResult CreateStudent(Student student)
69 {
70     if (student.Id <= 0)
71     {
72         return BadRequest(
73             "Student ID must be greater than zero."
74         );
75     }
76
77     if (string.IsNullOrEmpty(student.Name))
78     {
79         return BadRequest(
80             "Student name is required."
81         );
82     }
83
84     bool idExists = Students.Any(
85         existingStudent =>
86             existingStudent.Id == student.Id
87     );
88
89     if (idExists)
90     {
91         return BadRequest(
92             "A student with this ID already exists."
93         );
94     }
95
96     Students.Add(student);
97
98     logger.LogInformation(
99         "Student {StudentName} was created.",
100         student.Name
101     );
```

```

103         return CreatedAtAction(
104             nameof(GetStudent),
105             new { id = student.Id },
106             student
107         );
108     }

```

Output:



3. Return Appropriate HTTP Status Codes

Code:

```

63         return Ok(student);
64     }

```

```

103     return CreatedAtAction(
104         nameof(GetStudent),
105         new { id = student.Id },
106         student
107     );

```

```

70     if (student.Id <= 0)
71     {
72         return BadRequest(
73             "Student ID must be greater than zero."
74         );
75     }

```

```

51     if (student == null)
52     {
53         _logger.LogWarning(
54             "Student with ID {StudentId} was not found.",
55             id
56         );
57
58         return NotFound(
59             $"Student with ID {id} was not found."
60         );
61     }

```

Output:

The screenshot shows the Swagger UI interface in a web browser at localhost:5150/swagger/index.html. The selected endpoint is GET /api/Students/{id}. The parameter 'id' is required and is an integer (int32) in the path, with the value '1' entered in the input field. The 'Execute' button has been clicked, and the response is displayed. The response is a 200 status code with a JSON body: {"id": 1, "name": "Ali", "department": "Computer Science"}. The 'Curl' section shows the command: curl -X 'GET' \ 'http://localhost:5150/api/Students/1' \ -H 'accept: */*'. The 'Request URL' is http://localhost:5150/api/Students/1. The 'Server response' section shows the status code 200 and the response body.

department: Computer Science

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 26 Aug 2026 20:54:37 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

localhost:5150/swagger/index.html

POST /api/Students

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "id": 3,
  "name": "Ahmed",
  "department": "Artificial Intelligence"
}
```

Execute Clear

Responses

Curl

Android Studio

localhost:5150/swagger/index.html

Curl

```
curl -X 'POST' \
  'http://localhost:5150/api/Students' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 3,
    "name": "Ahmed",
    "department": "Artificial Intelligence"
  }'
```

Request URL

http://localhost:5150/api/Students

Server response

Code	Details
201	Response body

Undocumented

```
{
  "id": 3,
  "name": "Ahmed",
  "department": "Artificial Intelligence"
}
```

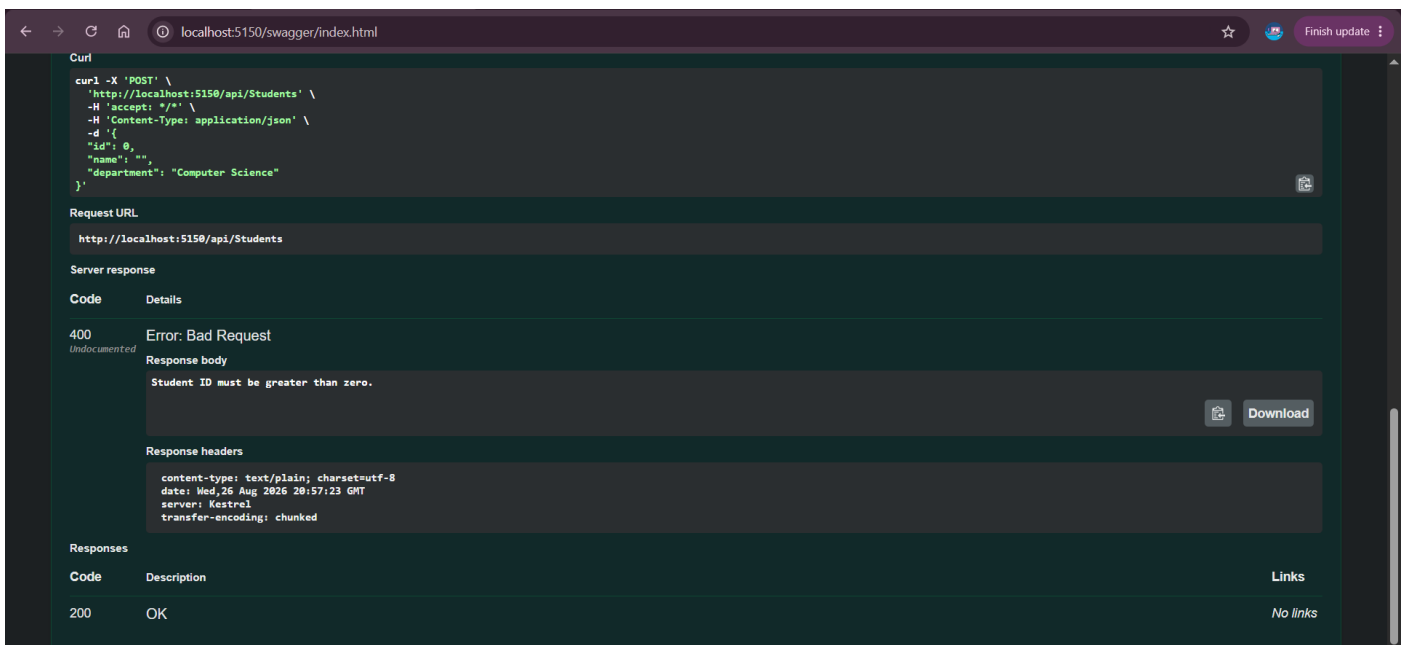
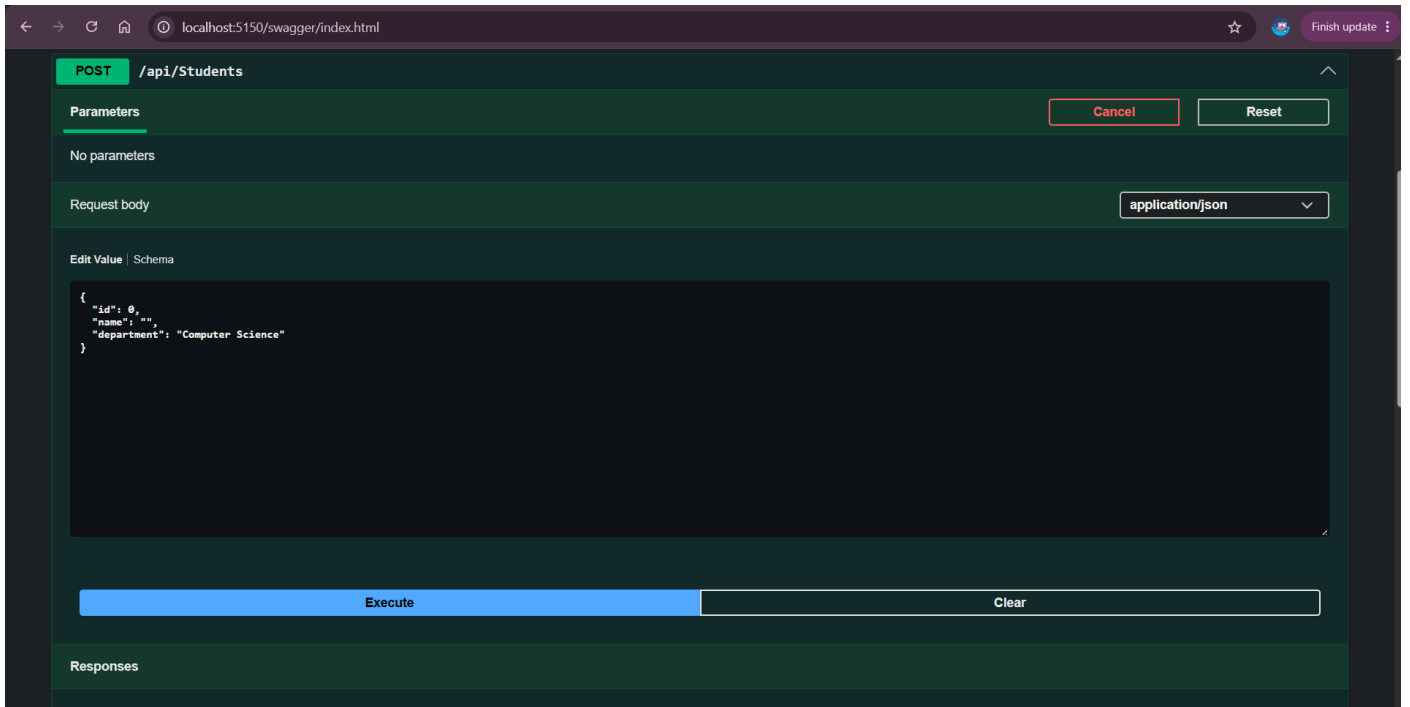
Response headers

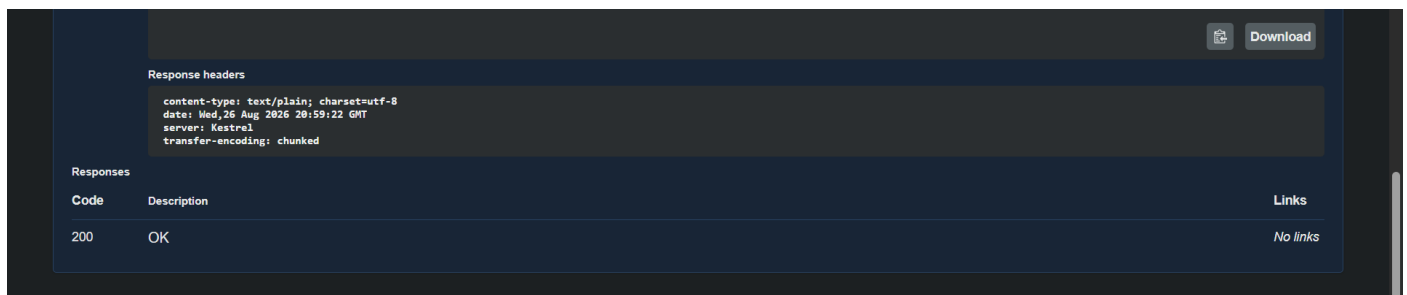
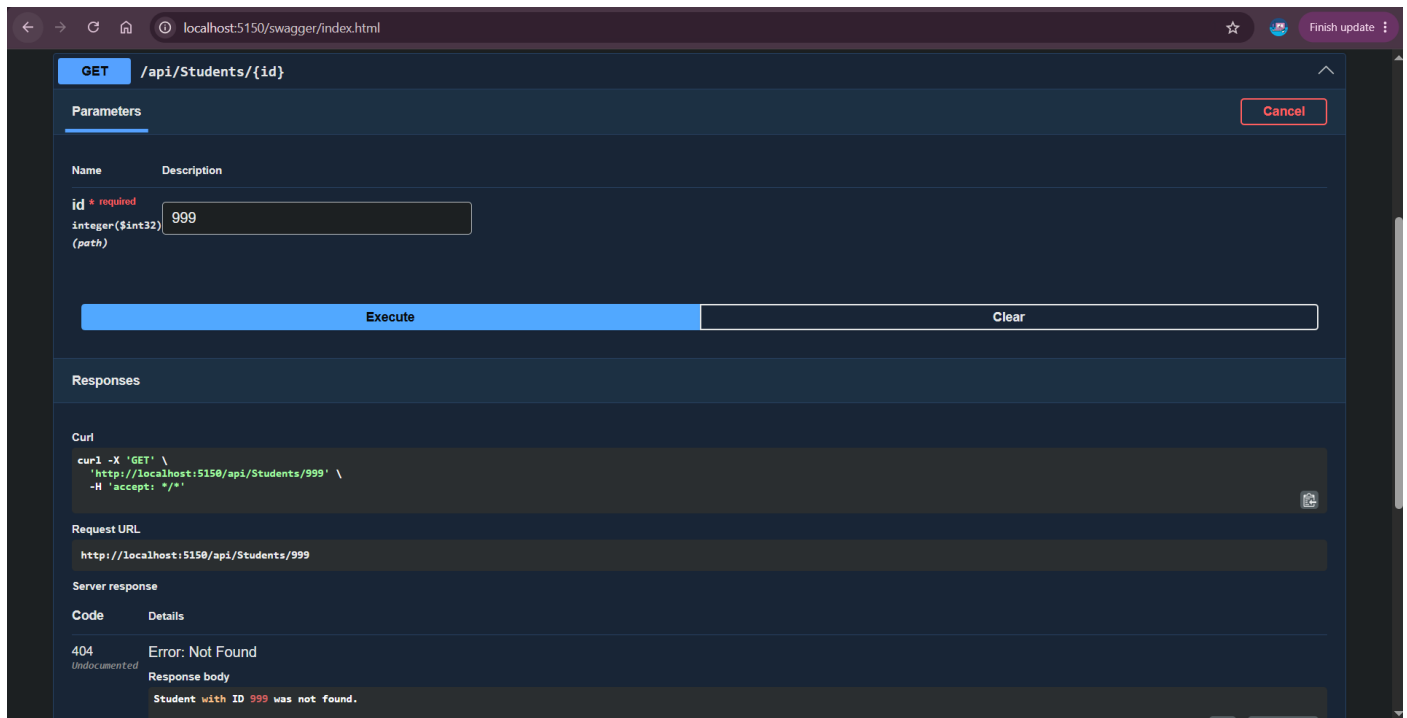
```
content-type: application/json; charset=utf-8
date: Wed, 26 Aug 2026 20:52:59 GMT
location: http://localhost:5150/api/Students/3
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

GET /api/Students/{id}





4. Test Every Endpoint Using Swagger and Postman

Code:

```

PartB-ASPNetCore-Basics > Controllers > StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers > StudentsController
1  using Microsoft.AspNetCore.Mvc;
2  using PartB_ASPNetCore_Basics.Models;
3
4  namespace PartB_ASPNetCore_Basics.Controllers;
5
6  [ApiController]
7  [Route("api/[controller]")]
8  public class StudentsController : ControllerBase
9  {
10     private static readonly List<Student> Students = new()
11     {
12         new Student
13         {
14             Id = 1,
15             Name = "Ali",
16             Department = "Computer Science"
17         },
18         new Student
19         {
20             Id = 2,
21             Name = "Sara",
22             Department = "Software Engineering"
23         }
24     };
25
26     private readonly ILogger<StudentsController> _logger;
27
28     public StudentsController(
29         ILogger<StudentsController> logger)
30     {
31         _logger = logger;
32     }
33
34

```



```

PartB-ASPNetCore-Basics > Controllers > C# StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers > StudentsController
4  namespace PartB_ASPNetCore_Basics.Controllers;
8  public class StudentsController : ControllerBase

36  [HttpGet]
    0 references
37  public IActionResult GetStudents()
38  {
39      _logger.LogInformation("Getting all students.");
40
41      return Ok(Students);
42  }
43
44
45  [HttpGet("{id}")]
    1 reference
46  public IActionResult GetStudent(int id)
47  {
48      Student? student = Students
49          .FirstOrDefault(student => student.Id == id);
50
51      if (student == null)
52      {
53          _logger.LogWarning(
54              "Student with ID {StudentId} was not found.",
55              id
56          );
57
58          return NotFound(
59              $"Student with ID {id} was not found."
60          );
61      }
62
63      return Ok(student);
64  }
65
66

```

```

PartB-ASPNetCore-Basics > Controllers > StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers > StudentsController
4 namespace PartB_ASPNetCore_Basics.Controllers;
8 public class StudentsController : ControllerBase

67 [HttpPost]
0 references
68 public IActionResult CreateStudent(Student student)
69 {
70     if (student.Id <= 0)
71     {
72         return BadRequest(
73             "Student ID must be greater than zero."
74         );
75     }
76
77     if (string.IsNullOrEmpty(student.Name))
78     {
79         return BadRequest(
80             "Student name is required."
81         );
82     }
83
84     bool idExists = Students.Any(
85         existingStudent =>
86             existingStudent.Id == student.Id
87     );
88
89     if (idExists)
90     {
91         return BadRequest(
92             "A student with this ID already exists."
93         );
94     }
95
96     Students.Add(student);
97
98     _logger.LogInformation(
99         "Student {StudentName} was created.",
100         student.Name
101     );

```

```

PartB-ASPNetCore-Basics > Controllers > StudentsController.cs > {} PartB_ASPNetCore_Basics.Controllers > StudentsController
4 namespace PartB_ASPNetCore_Basics.Controllers;
8 public class StudentsController : ControllerBase
68     public IActionResult CreateStudent(Student student)

102
103         return CreatedAtAction(
104             nameof(GetStudent),
105             new { id = student.Id },
106             student
107         );
108     }
109 }

```

Output:

localhost:5150/swagger/index.html

Swagger
Supported by SMARTBEAR

Select a definitionPartB-ASPNetCore-Basics v1

PartB-ASPNetCore-Basics1.0OAS 3.0
<http://localhost:5150/swagger/v1/swagger.json>

Students

GET/api/Students

POST/api/Students

GET/api/Students/{id}

WeatherForecast

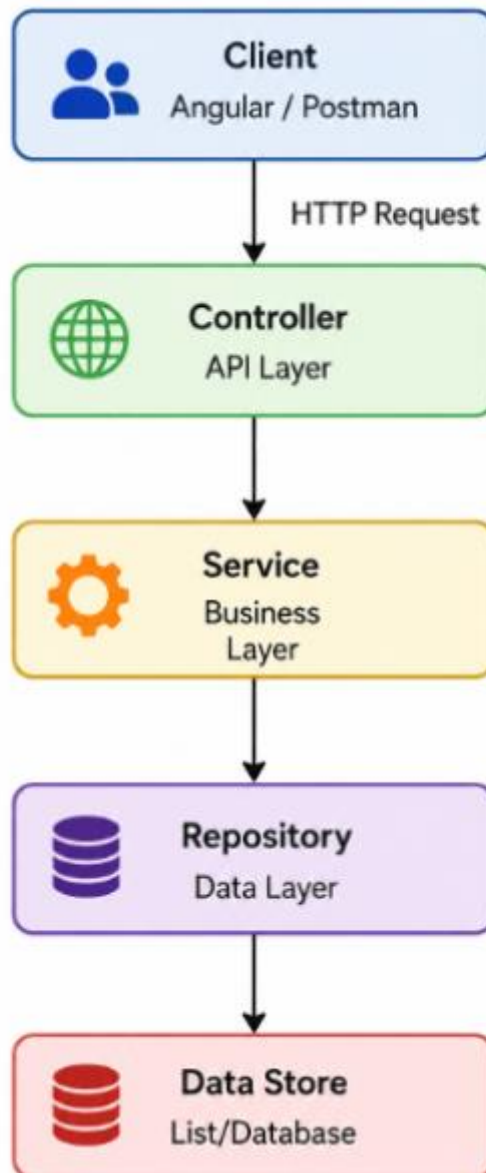
GET/WeatherForecast

Schemas

PART C: ARCHITECTURE CONCEPTS

Exercises:-

1. Controller → Service → Repository → Data Store Diagram



2. Responsibilities of Each Layer

Controller / Presentation Layer:

The Controller receives HTTP requests from the client and returns HTTP responses. It should handle routing, basic request validation, status codes, and communication with the service layer.

For example;

GET /api/books



BooksController

Service / Business Layer:

The Service contains business logic and application rules. The controller calls the service instead of directly working with the data store.

For example;

BookService could decide whether a book is valid before adding it.

Repository / Data Layer:

The Repository is responsible for reading and writing data. It hides the details of how data is stored from the service and controller.

For example;

BookRepository



List<Book>

Later:

BookRepository



Database

Data Store:

The Data Store is where application data is actually stored.

For example;

List<Book>

for your Week 2 project.

Later:

SQL Server

3. Why Database Code Should Not Be Placed Directly in Controllers

Putting database code directly inside controllers is poor design because controllers should mainly handle HTTP requests and responses. If database logic is placed in every controller, the code becomes difficult to maintain, test, and reuse. The same database operations may also be repeated in several places. Using a Service and Repository keeps responsibilities separate and makes it easier to change the data source without changing the controller.

For example;

Poor structure:

BooksController

- |— HTTP request handling
- |— validation
- |— business logic
- |— database connection

└─ SQL queries
└─ response handling

The controller is doing too much.

Better structure:

BooksController



BookService



BookRepository



Database

Each class has a clear responsibility.

PART D: ANGULAR BASICS FOR REAL APPLICATIONS

Exercises:-

1. Create a Reusable StudentService With Mock Data

Code:

```
PartD-Angular-Basics > src > app > TS student.ts > Student
1  export interface Student {
2      id: number;
3      name: string;
4      email: string;
5      department: string;
6  }
```

PartD-Angular-Basics > src > app > services > TS student.service.ts >  StudentService

```
1  import { Injectable } from '@angular/core';
2  import { Student } from '../student';
3
4  @Injectable({
5    providedIn: 'root'
6  })
7  export class StudentService {
8
9    private students: Student[] = [
10     {
11       id: 1,
12       name: 'Ali',
13       email: 'ali@example.com',
14       department: 'Computer Science'
15     },
16     {
17       id: 2,
18       name: 'Sara',
19       email: 'sara@example.com',
20       department: 'Software Engineering'
21     },
22     {
23       id: 3,
24       name: 'Ahmed',
25       email: 'ahmed@example.com',
26       department: 'Artificial Intelligence'
27     }
28   ];
29
30   getStudents(): Student[] {
31     return this.students;
32   }
33 }
```

PartD-Angular-Basics > src > app > student-list > TS student-list.ts >  StudentList

```
11  export class StudentList {
15    constructor(
16      private studentService: StudentService
17    ) {
18      this.students = this.studentService.getStudents();
19    }
20  }
```

Output:



Angular Student Application

[Students](#) [Registration](#)

Student List

ID	Name	Email	Department
1	Ali	ali@example.com	Computer Science
2	Sara	sara@example.com	Software Engineering
3	Ahmed	ahmed@example.com	Artificial Intelligence

2. Create a Registration Form Using Reactive Forms

Code:


```
15 export class Registration {
17   registrationForm = new FormGroup({
18     name: new FormControl('', [
19       Validators.required,
20       Validators.minLength(3)
21     ]),
22     email: new FormControl('', [
23       Validators.required,
24       Validators.email
25     ]),
26     password: new FormControl('', [
27       Validators.required,
28       Validators.minLength(6)
29     ])
30   });
31   onSubmit(): void {
32     if (this.registrationForm.valid) {
33       console.log(
34         this.registrationForm.value
35       );
36       alert('Registration successful.');
```

```
1  <h2>Student Registration</h2>
2
3  <form
4    [formGroup]="registrationForm"
5    (ngSubmit)="onSubmit()">
6
7    <div>
8
9      <label for="name">
10        Name
11      </label>
12
13      <input
14        id="name"
15        type="text"
16        formControlName="name">
17
18      @if (
19        registrationForm.controls.name.touched &&
20        registrationForm.controls.name.invalid
21      ) {
22
23        <p class="error">
24
25          @if (
26            registrationForm.controls.name.errors?.['required']
27          ) {
28            Name is required.
29          }
30
31          @if (
32            registrationForm.controls.name.errors?.['minlength']
33          ) {
34            Name must contain at least 3 characters.
35          }
36
37        </p>
38
```

```
3   <form
7     <div>

39   }
40
41   </div>
42
43
44   <div>
45
46     <label for="email">
47       Email
48     </label>
49
50     <input
51       id="email"
52       type="email"
53       formControlName="email">
54
55     @if (
56       registrationForm.controls.email.touched &&
57       registrationForm.controls.email.invalid
58     ) {
59
60       <p class="error">
61
62         @if (
63           registrationForm.controls.email.errors?.['required']
64         ) {
65           Email is required.
66         }
67
68         @if (
69           registrationForm.controls.email.errors?.['email']
70         ) {
71           Please enter a valid email address.
72         }
73
74       </p>
```

```
3    <form
44    <div>
75
76    }
77
78    </div>
79
80
81    <div>
82
83    <label for="password">
84    | Password
85    </label>
86
87    <input
88    | id="password"
89    | type="password"
90    | formControlName="password">
91
92    @if (
93    | registrationForm.controls.password.touched &&
94    | registrationForm.controls.password.invalid
95    ) {
96
97    <p class="error">
98
99    | @if (
100    | | registrationForm.controls.password.errors?.['required']
101    | ) {
102    | | Password is required.
103    | }
104
105    | @if (
106    | | registrationForm.controls.password.errors?.['minlength']
107    | ) {
108    | | Password must contain at least 6 characters.
109    | }
110
```

```

PartD-Angular-Basics > src > app > registration > registration.html > form
3  <form
81  <div>
97  <p class="error">
110
111  </p>
112
113  }
114
115  </div>
116
117
118  <button type="submit">
119    Register
120  </button>
121
122  </form>

```

Output:



Angular Student Application

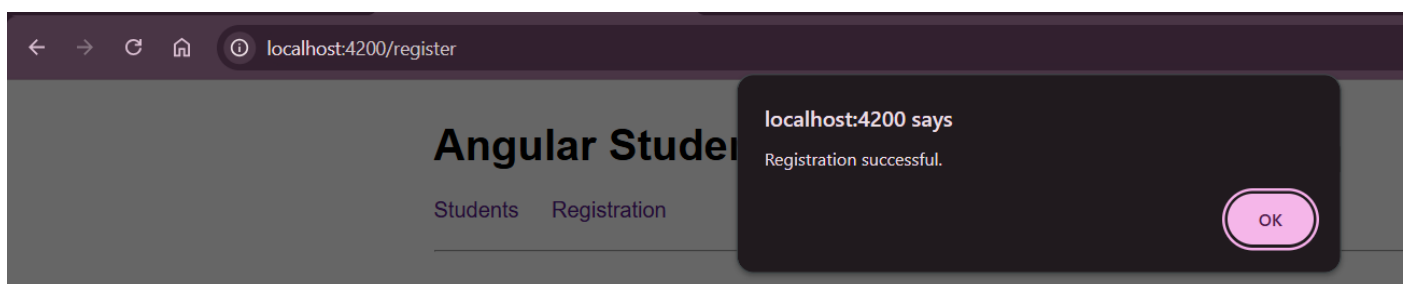
[Students](#) [Registration](#)

Student Registration

Name

Email

Password



3. Add Required, Email, and Minimum-Length Validation

Code:

```
PartD-Angular-Basics > src > app > registration > TS registration.ts > Registration > onSubmit
1  import { Component } from '@angular/core';
2  import {
3    FormControl,
4    FormGroup,
5    ReactiveFormsModule,
6    Validators
7  } from '@angular/forms';
8
9  @Component({
10   selector: 'app-registration',
11   imports: [ReactiveFormsModule],
12   templateUrl: './registration.html',
13   styleUrls: ['./registration.css']
14 })
15 export class Registration {
16
17   registrationForm = new FormGroup({
18
19     name: new FormControl('', [
20       Validators.required,
21       Validators.minLength(3)
22     ]),
23
24     email: new FormControl('', [
25       Validators.required,
26       Validators.email
27     ]),
28
29     password: new FormControl('', [
30       Validators.required,
31       Validators.minLength(6)
32     ])
33
34   });
```

```

18     @if (
19         | registrationForm.controls.name.touched &&
20         | registrationForm.controls.name.invalid
21     ) {
22
23         <p class="error">
24
25             @if (
26                 | registrationForm.controls.name.errors?.['required']
27             ) {
28                 | Name is required.
29             }
30
31             @if (
32                 | registrationForm.controls.name.errors?.['minlength']
33             ) {
34                 | Name must contain at least 3 characters.
35             }

```

```

55     @if (
56         | registrationForm.controls.email.touched &&
57         | registrationForm.controls.email.invalid
58     ) {
59
60         <p class="error">
61
62             @if (
63                 | registrationForm.controls.email.errors?.['required']
64             ) {
65                 | Email is required.
66             }
67
68             @if (
69                 | registrationForm.controls.email.errors?.['email']
70             ) {
71                 | Please enter a valid email address.
72             }

```

```

92     @if (
93         registrationForm.controls.password.touched &&
94         registrationForm.controls.password.invalid
95     ) {
96
97         <p class="error">
98
99             @if (
100                 registrationForm.controls.password.errors?.['required']
101             ) {
102                 Password is required.
103             }
104
105             @if (
106                 registrationForm.controls.password.errors?.['minlength']
107             ) {
108                 Password must contain at least 6 characters.
109             }

```

Output:

Angular Student Application

[Students](#) [Registration](#)

Student Registration

Name

Name is required.

Email

Please enter a valid email address.

Password

Password must contain at least 6 characters.

Register

4. Create Two Routes and Navigate Between Them

Code:

PartD-Angular-Basics > src > app > TS app.routes.ts > ...

```
1  import { Routes } from '@angular/router';
2  import { StudentList } from './student-list/student-list';
3  import { Registration } from './registration/registration';
4
5  export const routes: Routes = [
6
7      {
8          path: 'students',
9          component: StudentList
10     },
11
12     {
13         path: 'register',
14         component: Registration
15     },
16
17     {
18         path: '',
19         redirectTo: 'students',
20         pathMatch: 'full'
21     }
22
23 ];
```

PartD-Angular-Basics > src > app > app.html > div.container

```
1  <div class="container">
2
3  <h1>
4  |   Angular Student Application
5  </h1>
6
7  <nav>
8
9  |   <a routerLink="/students">
10 |     Students
11 |   </a>
12
13 |   <a routerLink="/register">
14 |     Registration
15 |   </a>
16
17 </nav>
18
19 <hr>
20
21 <router-outlet></router-outlet>
22
23 </div>
```

Output:



Angular Student Application

[Students](#) [Registration](#)

Student List

ID	Name	Email	Department
1	Ali	ali@example.com	Computer Science
2	Sara	sara@example.com	Software Engineering
3	Ahmed	ahmed@example.com	Artificial Intelligence

Angular Student Application

[Students](#) [Registration](#)

Student Registration

Name

Email

Password

Register